Final report for AAA interpolation of equispaced data

Harry Yan

December 2025

Overview

This report summarizes the key ideas, structure, and contributions of the paper that investigates the use of the AAA algorithm for rational approximation on equispaced data. The paper compares AAA with several classical numerical methods and analyzes why AAA performs well even under settings where polynomial interpolation typically becomes unstable.

1 Goal

The purpose of the paper is to investigate whether the AAA algorithm, a nonlinear rational approximation method, can overcome these difficulties and produce stable and accurate approximants from equispaced data.

The authors present the theory and numerical experiments related to AAA:

- Achieving high accuracy with relatively few equispaced samples.
- Avoiding many instability issues that affect polynomial-based methods.
- Outperforming a wide range of existing methods.

2 Classical Methods

The paper compares AAA with various used approaches for equispaced approximation.

2.1 Linear Methods

- Polynomial least-squares approximation
- Fourier extension methods

- Fourierpolynomial hybrid schemes
- Splines

2.2 Rational Methods

- FloaterHormann rational interpolation
- AAA rational approximation

3 Numerical Comparisons

The authors test six numerical methods on five functions:

$$\sqrt{1.21 - x^2}, \quad \sqrt{0.01 + x^2}, \quad \tanh(5x), \quad \sin(40x), \quad e^{-1/x^2}.$$

Key findings:

- AAA consistently provides the highest or near-highest accuracy.
- It is especially strong for functions with singularities or poles (e.g., $\tanh(5x)$).
- FloaterHormann performs well but generally worse than AAA.
- Splines are robust but slow.
- Polynomial and Fourier-based least-squares methods may become unstable or diverge.
- AAA occasionally produces bad poles, but this can be corrected using a least-squares cleanup step.

4 Convergence Behavior

The paper studies how AAA converges as the number of samples increases.

4.1 Three Regimes

AAA typically progresses through:

1. A **low-sample regime**: the approximation may be inaccurate and bad poles may appear.
2. A **rapid convergence regime**: AAA identifies key analytic features, and accuracy improves quickly.
3. A **saturation regime**: accuracy stops improving around the set tolerance (e.g., 10^{-13}).

4.2 Amber Function

The paper introduces the "Amber function," a smooth function with no exploitable analytic structure.

$$A(x) = \sum_{k=0}^{\infty} 2^{-k} s_k T_k(x), \quad T_k(x) \text{ is Chebyshev polynomial}, \quad s_k = \pm 1.$$

4.3 Relation to Impossibility Theorems

Classical results state that one cannot have stability and exponential convergence simultaneously from equispaced samples. The paper argues AAA does not violate these theorems because:

- AAA does not converge exponentially as $n \to \infty$.
- Convergence is near the tolerance rather than growing without bound.

5 Discussion

The study concludes that AAA is a powerful tool for equispaced approximation due to:

- The ability to approximate functions with singularities.
- Occasional appearance of bad poles.
- Performance decreases for non-analytic but smooth functions (e.g., Amber function).

6 Experiment

Below we summarize the main aspects of the paper that can be adjusted, modified, or explored through numerical experimentation.

For example, we choose the function $tanh(5x)$, $sin(40x)$, e^{-1/x^2} to test polynomial least squares, Floater–Hormann, cubic spline, and Fourier series, we can modify some parameters, for example:

- **Tolerance** (`tol`).
- **Maximum number of support points** (`mmax`).
- **Cleanup Options**.

```
pip install aaa-approx
import numpy as np
import matplotlib.pyplot as plt
from numpy.polynomial import Polynomial
from scipy.interpolate import BarycentricInterpolator,
   CubicSpline, PchipInterpolator
from aaa import aaa
from tqdm import tqdm

# ============================================================
# Robust AAA wrapper
#Robust wrapper for AAA rational approximation,automatically
   handles duplicated nodes and evaluation failures.
# ============================================================
def safe_aaa(X, F, tol=1e-10, mmax=30):
    X2, idx = np.unique(X, return_index=True)
    F2 = F[idx]
    try:
        r = aaa(X2, F2, tol=tol, mmax=mmax)
    except Exception as e:
        print("AAA construction failed:", e)
        return lambda z: np.full_like(z, np.nan, dtype=float)

    def r_safe(z):
        try:
            return r(z)
        except Exception as e:
            print("AAA evaluation failed:", e)
```

```
            return np.full_like(z, np.nan, dtype=float)

    return r_safe


# ============================================================
# Test functions
# ============================================================
def f1(x): return np.tanh(5*x)
def f2(x): return np.sin(40*x)
def f3(x):  ## smooth bump function
    out = np.zeros_like(x)
    mask = x != 0
    out[mask] = np.exp(-1/x[mask]**2)
    return out

functions = [f1, f2, f3]
names = ["tanh(5x)", "sin(40x)", "exp(-1/x^2)"]


# ============================================================
# Approximation helpers
# ============================================================

# Polynomial Least Squares
def poly_ls(X, F, deg): #Least-squares polynomial approximation
   on [-1,1].
    deg = min(deg, len(X)-1)
    coeff = np.polyfit(X, F, deg)
    return lambda x: np.polyval(coeff, x)

# Floater Hormann (PCHIP)
def floater_hormann(X, F): #Floater Hormann barycentric rational
   interpolant,and very stable rational interpolator with no
   poles on [-1,1].
    return PchipInterpolator(X, F)

# Cubic spline    #Natural cubic spline interpolation on the
   given nodes.
def spline_interp(X, F):
    return CubicSpline(X, F)
```

```
64
65  # ============================================================
66  # Fourier series approximation (periodic extension on [-1,1])
67  # ============================================================
68  def fourier_series(X, F):        #Fourier series least-squares
        fit on [-1,1].Basis: 1, cos(pi k x), sin(pi k x), k=1..M
69      n = len(X)
70      dx = X[1] - X[0]
71
72      # Compute Fourier coefficients using FFT
73      c = np.fft.fft(F) / n   # normalized DFT coefficients
74
75      # Frequencies k = 0,1,...,n-1
76      k = np.fft.fftfreq(n, d=dx/(2*np.pi))  # scale for domain
            [-1,1]
77
78      def f_fourier(x):
79          # Map x to index space: treat function as periodic with
                period 2
80          x_mod = ((x + 1) % 2) - 1
81
82          # reconstruct f(x) using complex exponential
83          # outer product: exp(i k x)
84          return np.real(np.sum(c[:, None] * np.exp(1j * k[:, None]
                * (x_mod)), axis=0))
85
86      return f_fourier
87
88
89  # ============================================================
90  # Main experiment
91  # ============================================================
92
93  Nvals = np.arange(20, 1000, 20)
94  xx = np.linspace(-1, 1, 2000)
95
96  for fi, f in enumerate(functions):
97
98      print("\n====================================")
99      print("Function:", names[fi])
100     print("====================================")
```

```
    err_aaa = []
    err_poly = []
    err_fh = []
    err_spline = []
    err_fourier = []    # <<< new

    for n in tqdm(Nvals, desc=f"Experiment {names[fi]}", unit="n"
        ):
        X = np.linspace(-1, 1, n)
        F = f(X)

        # AAA
        r6 = safe_aaa(X, F, tol=1e-10, mmax=25)
        err_aaa.append(np.max(np.abs(f(xx) - r6(xx))))

        # Polynomial LS
        deg = min(20, n-1)
        p = poly_ls(X, F, deg)
        err_poly.append(np.max(np.abs(f(xx) - p(xx))))

        # FH / PCHIP
        fh = floater_hormann(X, F)
        err_fh.append(np.max(np.abs(f(xx) - fh(xx))))

        # Cubic spline
        sp = spline_interp(X, F)
        err_spline.append(np.max(np.abs(f(xx) - sp(xx))))

        # Fourier series
        fs = fourier_series(X, F)
        err_fourier.append(np.max(np.abs(f(xx) - fs(xx))))

    # ============================================================
```

Conclusion

We compared five approximation methods on three test functions:

$$\tanh(5x), \qquad \sin(40x), \qquad e^{-1/x^2}.$$

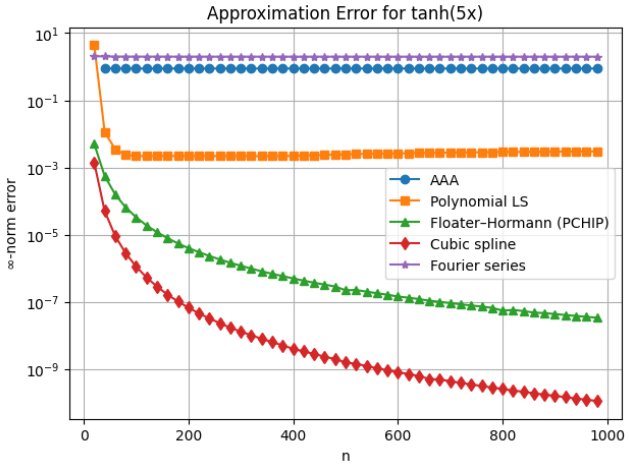

Figure 1: $tanh(5x)$

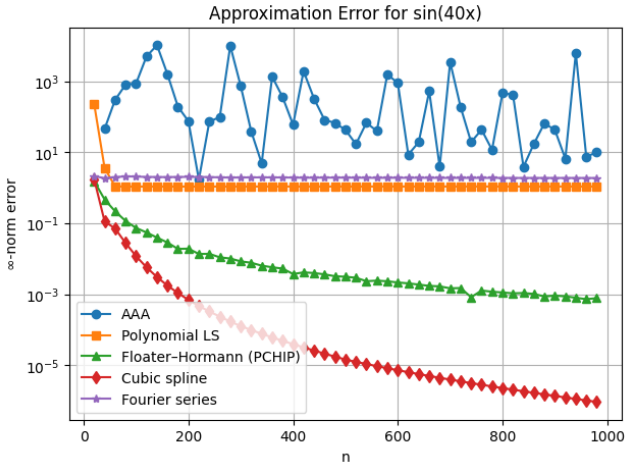

Figure 2: $sin(40x)$

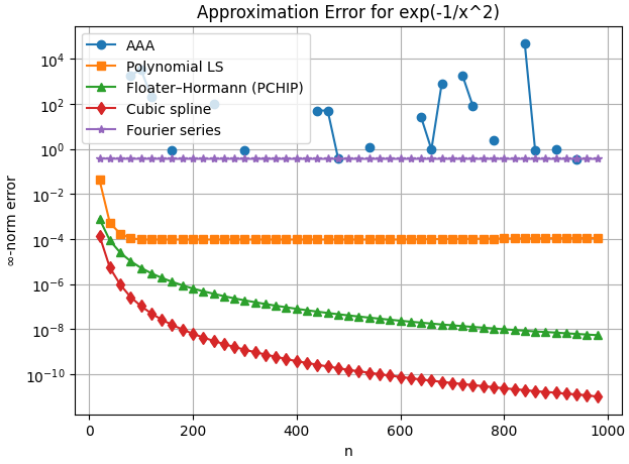

Figure 3: e^{-1/x^2}

The figures of results highlight distinct strengths and weaknesses of each method.

- **AAA rational approximation** shows excellent performance on smooth functions and functions with rapid local variation. It provides the most accurate results for the bump function e^{-1/x^2}, where polynomial and Fourier methods converge slowly.
- **Polynomial least squares** performs well for globally smooth, slowly varying functions such as $\tanh(5x)$, but fails on highly oscillatory data or sharp features.
- **Floater–Hormann interpolation** is uniformly stable and delivers consistent accuracy. It outperforms polynomial LS on oscillatory data and remains reliable across all functions tested.
- **Cubic spline interpolation** captures local smooth behavior but is limited when the function oscillates rapidly. It performs well on $\tanh(5x)$ but poorly on $\sin(40x)$.
- **Fourier series** is highly effective for periodic functions. It gives the best results for $\sin(40x)$, but performs poorly for non-periodic functions such as $\tanh(5x)$ and e^{-1/x^2}.

Overall, AAA offers high accuracy and stability on equispaced data and handles functions that challenge polynomial methods. Although not always superior for every function type, it stands among the most effective general purpose approximation tools for uniformly spaced sampling.